(1) Pandas

happiness2020.pkl

- country: *Name of the country*
- happiness_score: *Happiness score*
- social_support: *Social support (mitigation the effects of inequality)*
- healthy_life_expectancy: *Healthy Life Expectancy*
- freedom_of_choices: *Freedom to make life choices*
- generosity: *Generosity (charity, volunteers)*
- perception_of_corruption: *Corruption Perception*
- world_region: *Area of the world of the country*

countries_info.csv

- country_name: *Name of the country*
- area: *Area in sq mi*
- population: *Number of people*
- literacy: *Literacy percentage*

Read a CSV File

```
pd.read_csv(
    filepath_or_buffer,
    sep=",",
    header="infer",
    names=None,
    usecols=None,
    dtype=None,
    na_values=None,
    parse_dates=False,
    nrows=None,
    chunksize=None
)

# XML
df = pd.read_xml("data.xml", xpath=".//record")
print(df)


happiness = pd.read_csv(HAPPINESS_DATASET)
countries = pd.read_csv(COUNTRIES_DATASET, decimal=',')
```

Important parameters: `filepath_or_buffer` specifies the file path or URL; `sep` defines the column delimiter; `header` and `names` control how column names are read; `usecols` limits which columns are loaded; `dtype` enforces column data types; `na_values` defines which values are treated as missing; `parse_dates` enables automatic `datetime` parsing; and `nrows` or `chunksize` are commonly used to limit memory usage when working with large files.

Drop Columns/Rows

```
# DataFrame.drop(...)
happiness_clean = happiness.drop(
    labels=["perception_of_corruption"],  # column(s) or row label(s)
    axis=1,                               # 1 = columns, 0 = rows
    errors="ignore"                       # ignore if column missing
)

# drop rows by index label (example: first 3 rows)
happiness_no_first3 = happiness.drop(labels=happiness.index[:3], axis=0)
```

Drop / fill missing values (NaNs)

```
# DataFrame.dropna(...)
happiness_no_na = happiness.dropna(
    axis=0,           # drop rows with NA
    how="any",        # "any" drops if at least one NA; "all" drops only
if all are NA
    subset=["happiness_score", "healthy_life_expectancy"]
)

# DataFrame.fillna(...)
happiness_filled = happiness.fillna
(
    value={"perception_of_corruption":
happiness["perception_of_corruption"].median()},
    inplace=False
)

# quick NA check
na_counts = happiness.isna().sum()
```

Rename columns

```
# DataFrame.rename(...)
countries2 = countries.rename(
```

```
    columns={"country_name": "country"},
    inplace=False
)
```

`assign`, `astype`, `to_numeric`

```
import pandas as pd

# --- pd.to_numeric(arg, errors="raise", downcast=None) ---
# Turns a Series into numbers. errors="coerce" => invalid values become
NaN (instead of crashing).

# --- DataFrame.assign(**kwargs) ---
# Returns a NEW DataFrame with added/replaced columns (does NOT modify the
original).

# --- DataFrame.astype(dtype, copy=None, errors="raise") ---
# Changes dtype(s). Use {"col": "Int64"} for pandas' nullable integer
(supports missing values).

countries_clean = (
    countries
    .assign(
        population=pd.to_numeric(countries["population"],
errors="coerce"),  # parse to numeric (NaN if bad)
        area=pd.to_numeric(countries["area"], errors="coerce"),
        literacy=pd.to_numeric(countries["literacy"], errors="coerce"),
    )
    .astype({"population": "Int64"})  # convert population to nullable
integer dtype
)

countries_clean[["population", "area", "literacy"]].dtypes
```

Apply a function to a column (lambda or `def`)

```
# Series.apply(...)
happiness2 = happiness.copy()
happiness2["generosity_scaled"] = happiness2["generosity"].apply(
    lambda x: 0 if pd.isna(x) else max(x, 0)   # clamp negatives + handle
NA
)

def happiness_bucket(score: float) -> str:
    if pd.isna(score):
        return "unknown"
    if score >= 7:
```

```
        return "high"
    if score >= 5:
        return "mid"
    return "low"

happiness2["happiness_bucket"] =
happiness2["happiness_score"].apply(happiness_bucket)


def buckets(row):
    # outputs TWO values (2 columns)
    score = row["happiness_score"]
    life  = row["healthy_life_expectancy"]

    score_bucket = "unknown" if pd.isna(score) else ("high" if score >= 7
else "mid" if score >= 5 else "low")
    life_bucket  = "unknown" if pd.isna(life)  else ("high" if life >= 70
else "low")

    return pd.Series({"score_bucket": score_bucket, "life_bucket":
life_bucket})

happiness2[["score_bucket", "life_bucket"]] = happiness2.apply(buckets,
axis=1)
```

Create new columns (common patterns)

```
# 1) direct arithmetic
happiness2["support_per_life"] = happiness2["social_support"] /
happiness2["healthy_life_expectancy"]

# 2) np.where (fast conditional)
happiness2["is_high_happiness"] = np.where(happiness2["happiness_score"]
>= 7, 1, 0)

# 3) DataFrame.assign(...) (nice for chaining)
happiness3 = happiness.assign(
    corruption_inverted=lambda df: 1 - df["perception_of_corruption"]
)
```

Merge

```
happiness["country"] = happiness["country"].apply(lambda r: r.lower())

country_features = pd.merge(
    happiness,
```

```
    countries,
    how="inner",
    left_on="country",
    right_on="country"
)

#Same Key
out = pd.merge(left_df, right_df, on="key", how="inner")

# Different key
out = pd.merge(left_df, right_df, left_on="key_left",
right_on="key_right", how="inner")

# Multiple Keys
out = pd.merge(left_df, right_df, on=["key1", "key2"], how="left")
# or
out = pd.merge(left_df, right_df, left_on=["k1","k2"], right_on=
["r1","r2"], how="left")
```

Searching And Selecting

Usually they return a new DataFrame, unless you assign it to the old one.

```
df = happiness
```

1. Select columns only: `df[...]` and `.loc[:, ...]`

```
# one column (Series)
df["happiness_score"]

# multiple columns (DataFrame)
df[["country", "world_region", "happiness_score"]]

# same thing via loc (explicit row/col)
df.loc[:, ["country", "world_region", "happiness_score"]]
```

2. Select rows by label/position (slicing)

```
# first 5 rows by position
df.iloc[:5]

# rows 10..19 and specific columns
df.iloc[10:20, [0, 1, 2]]
```

3. Filter rows by condition + choose columns with `.loc`

```
df.loc[ROW_CONDITION, COLUMN_LIST]

df.loc[df["happiness_score"] >= 7, ["country", "happiness_score",
"world_region"]]

# Multiple conditions
df.loc[
    (df["world_region"] == "Western Europe") & (df["happiness_score"] >=
7),
    ["country", "happiness_score", "healthy_life_expectancy"]
]

mask = (df["happiness_score"] >= 7) & (df["world_region"] == "Western
Europe")
df.loc[mask, ["country", "world_region", "happiness_score"]]

# Use `&`, `|`, `~` (not `and/or/not`) and wrap each condition in
parentheses:
df.loc[
    (df["happiness_score"].between(6, 7.5)) &
    (df["world_region"].isin(["Western Europe", "North America and ANZ"]))
&
    (~df["country"].str.contains("United", case=False, na=False)),
    ["country", "world_region", "happiness_score"]
]

# Doing type conversion “inside” a `loc` filter
df.loc[
    pd.to_numeric(df["healthy_life_expectancy"], errors="coerce") >= 70,
    ["country", "healthy_life_expectancy"]
]

# startswith/endswith
df.loc[df["country"].str.startswith("A", na=False), ["country",
"happiness_score"]]

# extract pattern (e.g. text inside parentheses) and filter on it
code = df["country"].str.extract(r"\((.*?)\)", expand=False)
df.loc[code.notna(), ["country"]]
```

4. Membership filter: `.isin(...)`

```
regions = ["Western Europe", "North America and ANZ"]

df.loc[
```

```
    df["world_region"].isin(regions),
    ["country", "world_region", "happiness_score"]
]
```

5. Range filter: `.between(...)`

```
df.loc[
    df["happiness_score"].between(5, 7, inclusive="left"),
# "both,neither,left,right " inclusive
    ["country", "happiness_score"]
]
```

6. Text filter: `.str.contains(...)`

```
df.loc[
    df["country"].str.contains("land", case=False, na=False),
    ["country", "world_region", "happiness_score"]
]
```

7. SQL-like filter: `.query(...)`

```
df.query(
    "happiness_score >= 7 and world_region == 'Western Europe'"
)[["country", "world_region", "happiness_score"]]


### How it parses the string (important rules)

# Column names are treated like variables.
# You write comparisons with `==`, `>=`, etc.
# Combine conditions with `and`, `or`, `not` (or `&`, `|`, `~` also work
in many cases, but `and/or` is the intended style in `query`).
# Strings inside the query need quotes: `"world_region == 'Western
Europe'"`

# Using Python variables inside `query` with `@`
region = "Western Europe"
min_score = 7

df.query("world_region == @region and happiness_score >= @min_score")

# Membership
regions = ["Western Europe", "North America and ANZ"]
df.query("world_region in @regions")

# Ranges
df.query("happiness_score >= 5 and happiness_score < 7")
```

```
# don't try to convert inside query, will probably fail, first change the
col
df2 = df.assign(life_num=pd.to_numeric(df["healthy_life_expectancy"],
errors="coerce"))
df2.query("life_num >= 70")[["country", "life_num"]]

# With loc and mask
mask = df["country"].str.contains("land", case=False, na=False)
df.loc[mask].query("happiness_score >= 6")[["country", "happiness_score"]]

# Complex example

mask_name = df["country"].str.contains("land", case=False, na=False)

df.loc[mask_name].query(
    "happiness_score >= 6 and world_region != 'Sub-Saharan Africa'"
)[["country", "world_region", "happiness_score"]]
```

8. Combine row + column selection in one line

```
df.loc[df["world_region"] == "Western Europe", ["country",
"happiness_score"]] \
  .sort_values("happiness_score", ascending=False) \
  .head(10)
```

GroupBy and Aggregations

```
df = happiness
```

1. Basic `groupby` → one aggregation

```
# mean happiness per region
region_mean = df.groupby("world_region")["happiness_score"].mean()
region_mean.sort_values(ascending=False)
```

2. Multiple aggregations on one column (`.agg([...])`)

```
region_stats = (
    df.groupby("world_region")["happiness_score"]
      .agg(["count", "mean", "median", "min", "max"])
      .sort_values("mean", ascending=False)
```

```
)
region_stats
```

3. Multiple columns aggregated together

```
region_features = (
    df.groupby("world_region")[["happiness_score", "social_support",
"healthy_life_expectancy"]]
      .mean()
      .sort_values("happiness_score", ascending=False)
)
region_features
```

4. “Special” / custom aggregations with `agg` (named aggregations)

```
df.groupby(...).agg(
    OUTPUT_NAME = (INPUT_COLUMN, AGG_FUNCTION)
)

region_summary = (
    df.groupby("world_region")
      .agg(
          n_countries=("country", "count"),
          avg_happiness=("happiness_score", "mean"),
          avg_life=("healthy_life_expectancy", "mean"),
          std_happiness=("happiness_score", "std"),
      )
      .sort_values("avg_happiness", ascending=False)
)
region_summary
```

5. Different aggregations per column (dict-style)

```
region_mix = df.groupby("world_region").agg({
    "happiness_score": ["mean", "median"],
    "healthy_life_expectancy": ["mean", "min", "max"],
    "generosity": ["mean"]
})
region_mix

# To flatten the column names:

region_mix.columns = ["_".join(col).strip() for col in region_mix.columns]
region_mix = region_mix.reset_index()
```

```
region_mix.head()
```

6. Group by 2 columns → MultiIndex result

```
region_bucket_stats = (
    df.groupby(["world_region", "happiness_bucket"])["happiness_score"]
      .agg(["count", "mean"])
      .sort_values(["world_region", "mean"], ascending=[True, False])
)
region_bucket_stats
```

7. Accessing a MultiIndex result

```
# select one region from a MultiIndex result
region_bucket_stats.loc["Western Europe"]

# select one specific (region, bucket) pair
region_bucket_stats.loc[("Western Europe", "high")]
```

8. Convert MultiIndex rows to normal columns (`reset_index`)

```
region_bucket_stats_reset = region_bucket_stats.reset_index()
region_bucket_stats_reset.head()
```

9. Pivot-style view (wide table) from a MultiIndex groupby

```
wide = (
    df.groupby(["world_region", "happiness_bucket"])["happiness_score"]
      .mean()
      .unstack("happiness_bucket")   # move bucket from index into columns
)

wide

# Fill missing combinations:
wide_filled = wide.fillna(0)
```

10. `transform` : returns same length as original (adds group stats per row)

```
df2 = df.copy()
df2["region_avg_happiness"] = df2.groupby("world_region")
["happiness_score"].transform("mean")
df2["above_region_avg"] = df2["happiness_score"] >
df2["region_avg_happiness"]
```

```
df2[["country", "world_region", "happiness_score", "region_avg_happiness",
"above_region_avg"]].head()

# Use `transform` when you want a group statistic **attached back to each
row**

df.groupby("world_region")["happiness_score"].mean()
# If you have 10 regions, you get 10 values.

df["region_avg"] = df.groupby("world_region")
["happiness_score"].transform("mean")
# Each row gets “the mean happiness score of _its_ region”.
```

11. You can merge back your grouped info as well

```
group_features = (
    df.groupby("world_region")
      .agg(
          region_mean_score=("happiness_score", "mean"),
          region_std_score=("happiness_score", "std"),
          region_mean_life=("healthy_life_expectancy", "mean"),
          region_mean_support=("social_support", "mean"),
      )
      .reset_index()
)

df_enriched = df.merge(group_features, on="world_region", how="left")

#OR

df2 = df.copy()
df2["region_mean_score"]   = df2.groupby("world_region")
["happiness_score"].transform("mean")
df2["region_std_score"]    = df2.groupby("world_region")
["happiness_score"].transform("std")
df2["region_mean_life"]    = df2.groupby("world_region")
["healthy_life_expectancy"].transform("mean")
df2["region_mean_support"] = df2.groupby("world_region")
["social_support"].transform("mean")
```

Example : top N per group

```
top3_per_region = (
    df.sort_values("happiness_score", ascending=False)
```

```
        .groupby("world_region")
        .head(3)
        [["country", "world_region", "happiness_score"]]
)

top3_per_region
```

How to bin floats properly

`pd.cut` (equal-width bins)

```
df["age_bin"] = pd.cut(df["age"], bins=[0, 20, 30, 40, 50])
pd.crosstab(df["age_bin"], df["kids"])
```

`pd.qcut` (equal-frequency bins)

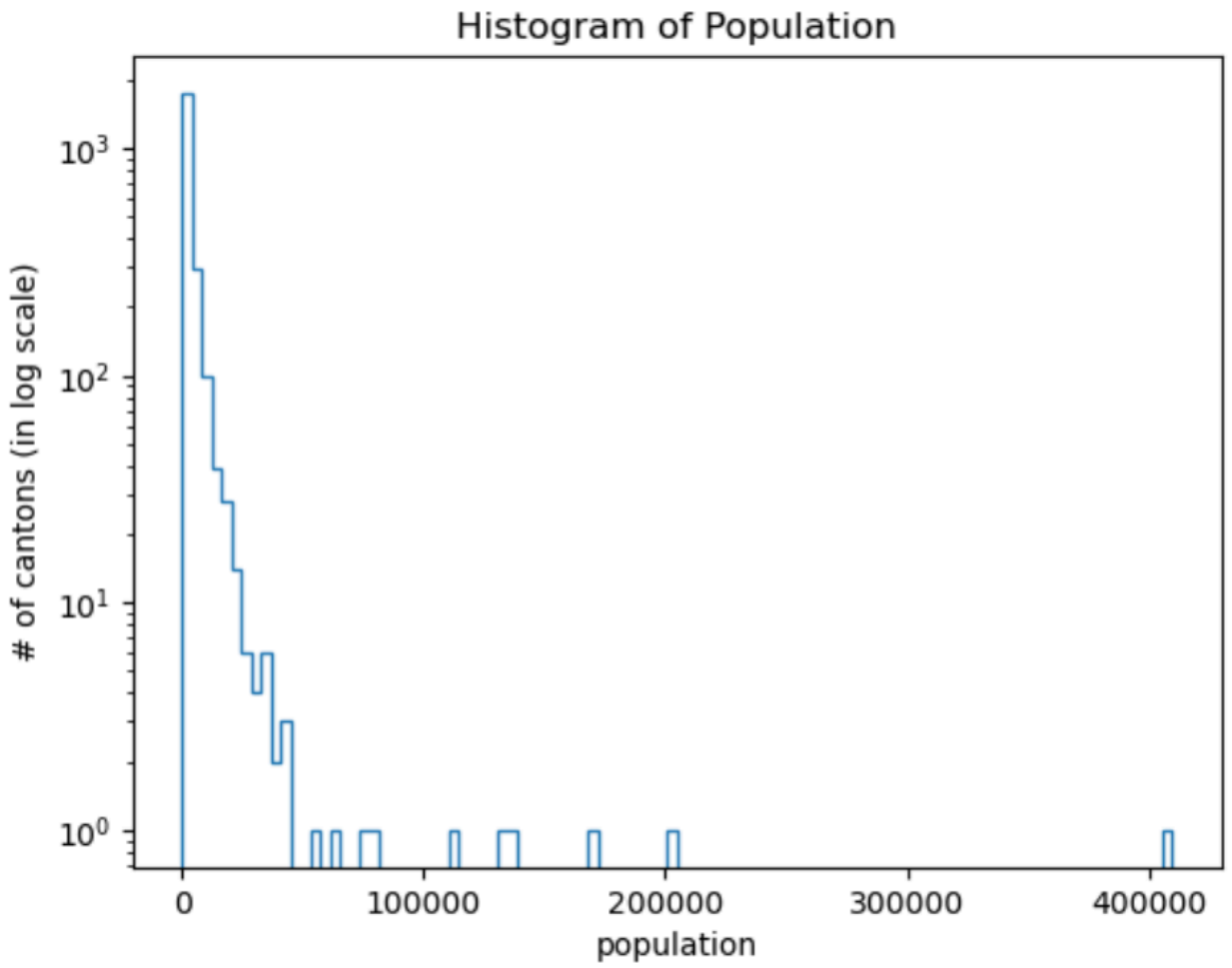
```
df["age_bin"] = pd.qcut(df["age"], q=4)
pd.crosstab(df["age_bin"], df["kids"])
```

Round floats (coarse grouping)

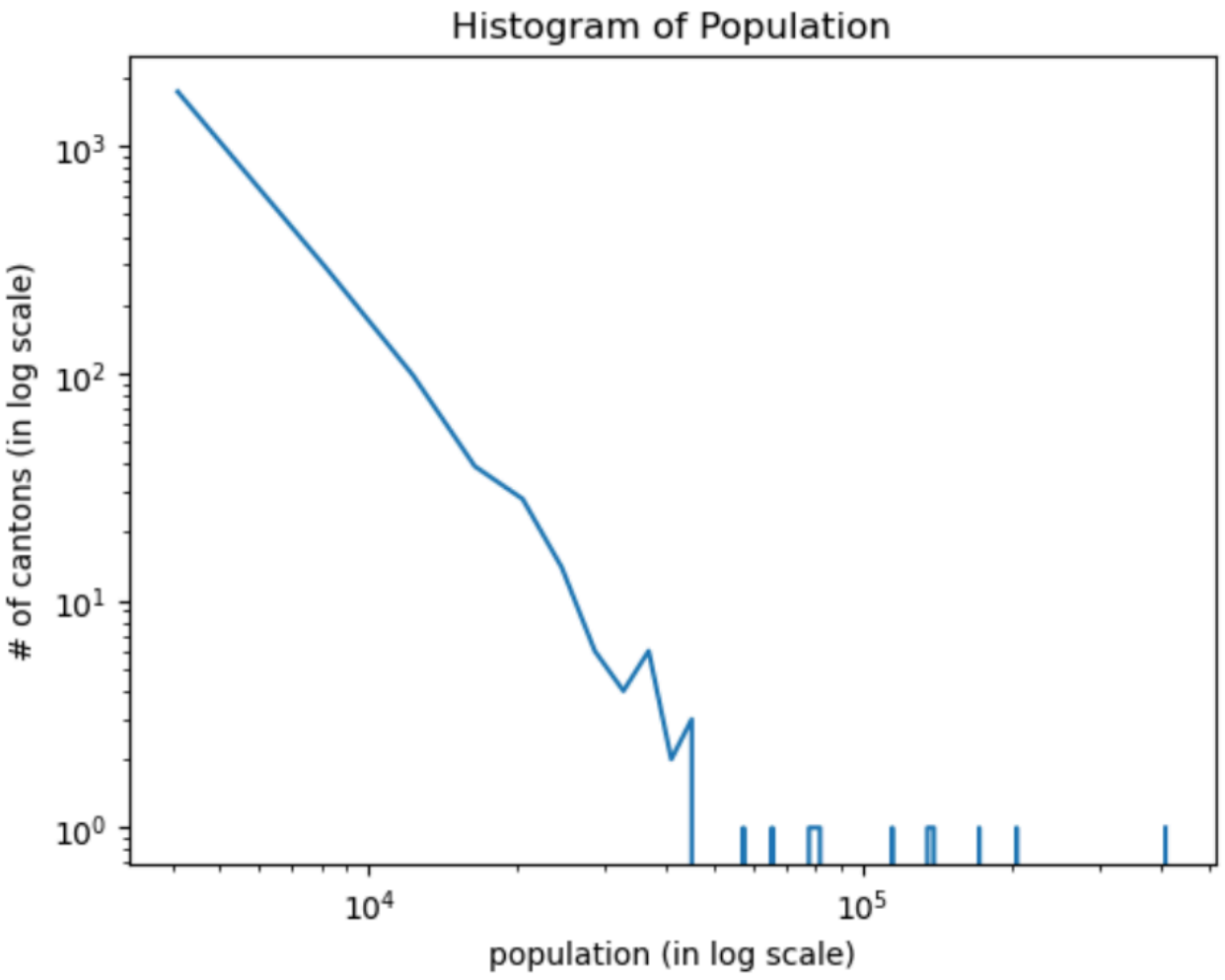
```
df["age_round"] = df["age"].round(0)
pd.crosstab(df["age_round"], df["kids"])
```

Power Law plots

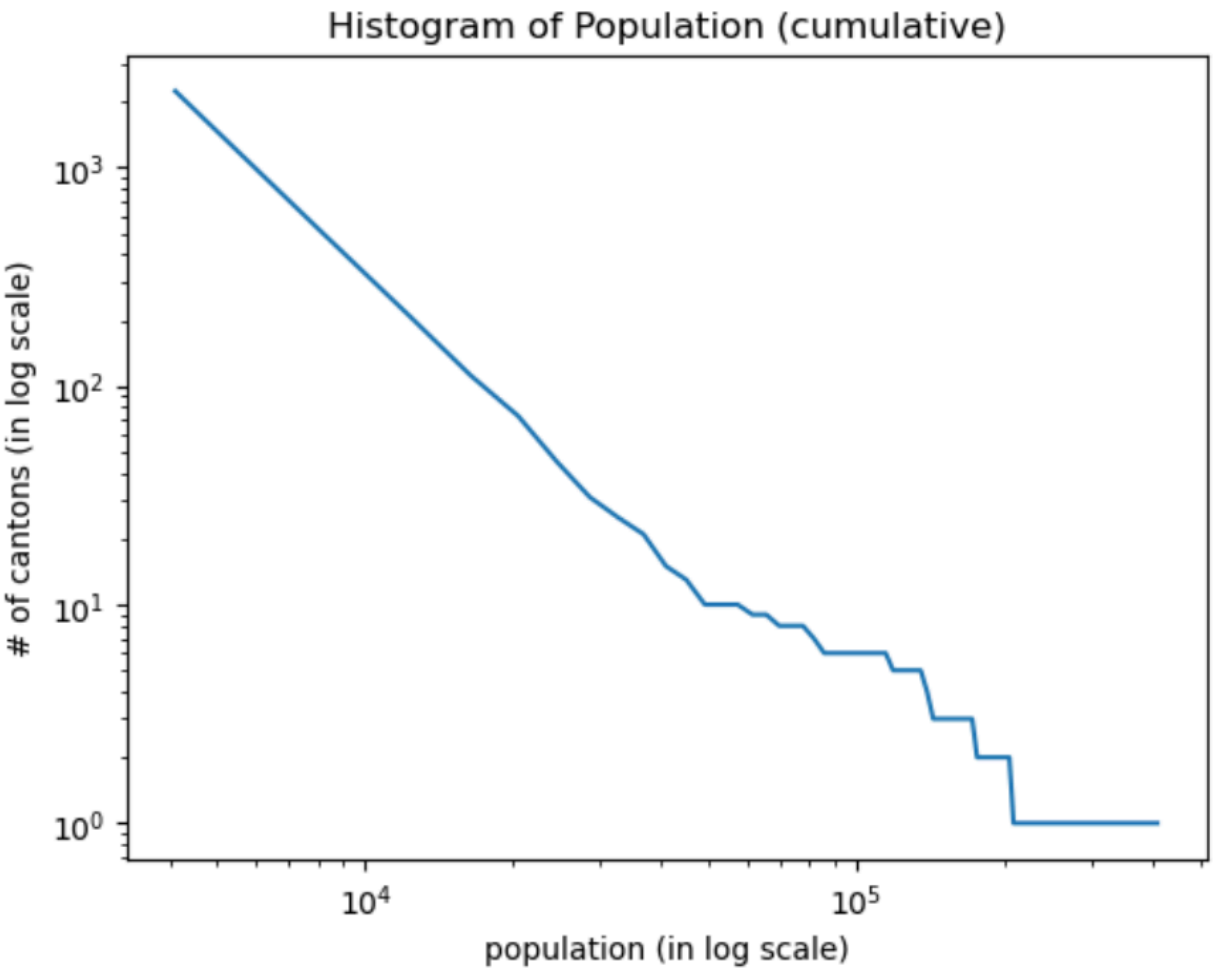
```
array_100 =
plt.hist(pop_per_commune.population_Dec,bins=100,log=True,histtype='step')
plt.title('Histogram of Population')
plt.ylabel('# of cantons (in log scale)')
plt.xlabel('population')
plt.show()
```




```
plt.loglog(array_100[1][1:],array_100[0])
plt.title('Histogram of Population')
plt.ylabel('# of cantons (in log scale)')
plt.xlabel('population (in log scale)')
plt.show()
```




```
array_cumulative=plt.hist(pop_per_commune.population_Dec,bins=100,log=True
,cumulative=-1,histtype='step')
plt.title('Histogram of Population (cumulative)')
plt.ylabel('# of cantons (in log scale)')
plt.xlabel('population')
plt.show()
```



Printing and accessing elements

```
def parse_conversation(line):
    season = line[0:3]
    episode = line[0:7]
    conversation = line[8:11]
    utterance = line[12:]
    return season,episode,conversation,utterance

target_line = "s10_e18_c11_u019"

print(parse_conversation(target_line))
df["season"] = df["conversation_id"].apply(lambda x: x[0:3])
df["episode"] = df["conversation_id"].apply(lambda x: x[0:7])

df["length"] = df["text"].apply(lambda x: len(x))

print("D. s10_e18_c11_u019: ", df.set_index("id").loc["s10_e18_c11_u019",
["season", "episode"]].values)
df.head()
```

```
print("E. s10_e18_c11_u019: ", df.set_index("id")
      .loc["s10_e18_c11_u019", "length"])

# ('s10', 's10_e18', 'c11', 'u019')
# D. s10_e18_c11_u019:  ['s10' 's10_e18']
# E. s10_e18_c11_u019:  17
```

GroupBy Apply

Group-wise aggregation with custom functions (`groupby().apply()`)

Use `groupby().apply()` when:

- you need **multiple columns at once**
- you need access to the **group keys**
- the aggregation logic is **not expressible** with simple `mean`, `sum`, etc.
  Unlike `agg`, **`apply` receives the entire group as a DataFrame**.

```
def my_func(group):
    # group is a DataFrame containing only rows of ONE group
    # group.name is the group key (tuple if multiple keys)
    ...
    return <something>

out = df.groupby(GROUP_KEYS).apply(my_func)
```

Return a `Series` → one row per group (most common)

```
def group_summary(group):
    # Access group keys
    # If grouping by multiple columns, group.name is a tuple
    region = group.name

    return pd.Series({
        "n_countries": len(group),
        "avg_happiness": group["happiness_score"].mean(),
        "avg_life": group["healthy_life_expectancy"].mean(),
        "support_to_life_ratio":
            group["social_support"].sum() /
group["healthy_life_expectancy"].sum(),
    })

region_custom = (
    happiness
```

```
    .groupby("world_region")
    .apply(group_summary)
    .sort_values("avg_happiness", ascending=False)
)

region_custom
```

2. Grouping by multiple columns (MultiIndex keys)

```
def bucket_stats(group):
    region, bucket = group.name  # unpack keys

    return pd.Series({
        "count": len(group),
        "mean_score": group["happiness_score"].mean(),
    })

stats = (
    happiness
    .groupby(["world_region", "happiness_bucket"])
    .apply(bucket_stats)
)

stats
```

3. Return a scalar → one column result

```
max_gap = happiness.groupby("world_region").apply(
    lambda g: g["happiness_score"].max() - g["happiness_score"].min()
)

max_gap
```

4) Return a DataFrame → multiple rows per group

This lets you **transform or filter groups**

Example: keep only countries above the group mean

```
def above_group_mean(group):
    mean_score = group["happiness_score"].mean()
    return group[group["happiness_score"] > mean_score]

above_mean = happiness.groupby("world_region").apply(above_group_mean)
```

```
above_mean.head()
```

Indices

```
"""
================================================================================
=====
PANDAS INDICES — WHAT THEY ARE, HOW TO INSPECT THEM, AND HOW TO CHANGE
THEM
================================================================================
=====

An INDEX in pandas is:
• A label for rows
• Used for alignment, selection, joins, and grouping
• NOT just a row number (though it often looks like one)

Think of the index as the "row key", not the data itself.

================================================================================
=====
"""

import pandas as pd

# Example DataFrame
df = pd.DataFrame({
    "country": ["A", "B", "C"],
    "happiness_score": [7.2, 5.8, 6.1],
    "world_region": ["Europe", "Asia", "Europe"]
})


# --------------------------------------------------------------------------------
-----
# 1) WHAT IS THE INDEX?
# --------------------------------------------------------------------------------
-----

"""
By default, pandas assigns a RangeIndex:
    0, 1, 2, ..., n-1

This is NOT a column.
It lives in df.index.
"""
```

```python
print(df)
print("Index:", df.index)
print("Index type:", type(df.index))


# ------------------------------------------------------------------------
-----
# 2) CHECKING / INSPECTING THE INDEX
# ------------------------------------------------------------------------
-----

# Basic properties
df.index
df.index.name
df.index.names                # for MultiIndex
df.index.dtype
df.index.is_unique
df.index.has_duplicates

# Convert index to list
list(df.index)

# Check if a label exists
"A" in df.index               # usually False unless explicitly set


# ------------------------------------------------------------------------
-----
# 3) SETTING A COLUMN AS INDEX
# ------------------------------------------------------------------------
-----

"""
Use set_index when:
• A column uniquely identifies rows
• You want fast lookup / joins
• You don't want the column duplicated
"""

df_by_country = df.set_index("country")

print(df_by_country)
print("Index:", df_by_country.index)

# Keep column as well
df_by_country_keep = df.set_index("country", drop=False)


# ------------------------------------------------------------------------
-----
```

```
# 4) ACCESSING ROWS BY INDEX
# ------------------------------------------------------------------------
-----

"""
Once a column is an index, use .loc to access rows.
"""

print(df_by_country.loc["A"])
print(df_by_country.loc["A", "happiness_score"])


# ------------------------------------------------------------------------
-----
# 5) RESETTING THE INDEX
# ------------------------------------------------------------------------
-----

"""
reset_index moves the index BACK into a column
and creates a new default RangeIndex.
"""

df_reset = df_by_country.reset_index()

print(df_reset)
print("Index after reset:", df_reset.index)

# Drop index instead of keeping it
df_reset_drop = df_by_country.reset_index(drop=True)


# ------------------------------------------------------------------------
-----
# 6) CHANGING INDEX VALUES
# ------------------------------------------------------------------------
-----

"""
You can overwrite index values directly.
Useful for relabeling, but be careful.
"""

df2 = df.copy()
df2.index = ["row1", "row2", "row3"]
print(df2)

# Rename index labels
df2 = df2.rename(index={"row1": "r1"})
```

```
# ------------------------------------------------------------------------
-----
# 7) SORTING BY INDEX
# ------------------------------------------------------------------------
-----

df_sorted = df_by_country.sort_index()
df_sorted_desc = df_by_country.sort_index(ascending=False)


# ------------------------------------------------------------------------
-----
# 8) MULTIINDEX (HIERARCHICAL INDEX)
# ------------------------------------------------------------------------
-----

"""
MultiIndex occurs when:
• groupby on multiple columns
• pivot tables
• stacking data
"""

df_multi = (
    df
    .set_index(["world_region", "country"])
)

print(df_multi)
print("Index type:", type(df_multi.index))
print("Index levels:", df_multi.index.names)

# Access MultiIndex rows
df_multi.loc[("Europe", "A")]
df_multi.loc["Europe"]


# ------------------------------------------------------------------------
-----
# 9) RESETTING PART OF A MULTIINDEX
# ------------------------------------------------------------------------
-----

# Reset one level
df_reset_one = df_multi.reset_index(level="world_region")

# Reset all levels
df_reset_all = df_multi.reset_index()
```

```
# ------------------------------------------------------------------------
-----
# 10) CHECKING INDEX ALIGNMENT (VERY IMPORTANT)
# ------------------------------------------------------------------------
-----

"""
Pandas aligns on index automatically during operations.
This is powerful — and dangerous if you don't expect it.
"""

s1 = pd.Series([1, 2, 3], index=["A", "B", "C"])
s2 = pd.Series([10, 20, 30], index=["B", "C", "D"])

print(s1 + s2)
# Result aligns by index, not position:
# A    NaN
# B    12
# C    23
# D    NaN


# ------------------------------------------------------------------------
-----
# 11) SET INDEX TEMPORARILY (COMMON PATTERN)
# ------------------------------------------------------------------------
-----

"""
Useful for one-off lookups without mutating the DataFrame.
"""

value = (
    df
    .set_index("country")
    .loc["A", "happiness_score"]
)

print("Happiness of A:", value)


# ------------------------------------------------------------------------
-----
# 12) WHEN TO USE / NOT USE INDEX
# ------------------------------------------------------------------------
-----

"""
✔ Use index when:
```

```
    • Column uniquely identifies rows
    • Frequent lookups / joins
    • Grouped / hierarchical data

❌ Avoid index when:
    • You need row order only
    • Index has no semantic meaning
    • You plan many resets / reassignments
"""



# ------------------------------------------------------------------------
-----
# 13) MENTAL MODEL (REMEMBER THIS)
# ------------------------------------------------------------------------
-----

"""
• Index = row labels, NOT data
• .loc works on labels, .iloc on positions
• reset_index() puts index back into columns
• MultiIndex = multiple grouping keys
• Pandas ALWAYS aligns by index, never by row order

If something looks weird:
→ Check the index first.
"""
```

Iterating over elements

```
"""
========================================================================
=====
ITERATING OVER ROWS IN A PANDAS DATAFRAME
========================================================================
=====

This section explains HOW and WHEN to iterate over rows in a DataFrame,
what pandas actually gives you when you do so, and what the alternatives
are.

IMPORTANT PRINCIPLE:
--------------------
• Pandas is optimized for *vectorized operations*
• Row-wise iteration is slower and should be a LAST RESORT
• But sometimes (complex logic, external APIs, graph construction,
```

```
debugging)
  iteration is unavoidable

This section shows:
1) iterrows() – most common, slow, but clear
2) itertuples() – faster, preferred when iterating
3) Iterating over selected columns only
4) Iterating over filtered rows
5) When NOT to iterate (vectorized alternatives)
6) Mental model to remember

================================================================================
=====
"""

import pandas as pd

# Example DataFrame
df = pd.DataFrame({
    "country": ["A", "B", "C"],
    "happiness_score": [7.2, 5.8, 6.1],
    "world_region": ["Europe", "Asia", "Europe"]
})



# ------------------------------------------------------------------------
-----
# 1) iterrows(): iterate row-by-row (index + Series)
# ------------------------------------------------------------------------
-----

"""
df.iterrows() yields:
    index, row
where:
    • index is the row label
    • row is a pandas Series (copy, not a view!)

Use case:
    • Simple logic
    • Debugging
    • Prototyping
"""

for idx, row in df.iterrows():
    print("Index:", idx)
    print("Country:", row["country"])
    print("Score:", row["happiness_score"])
    print("---")
```

```python
# ⚠️ Important:
# - row is a COPY → modifying it does NOT modify df
# - Very slow for large DataFrames


# ------------------------------------------------------------------------
-----
# 2) itertuples(): FAST row iteration (preferred if you must iterate)
# ------------------------------------------------------------------------
-----

"""
df.itertuples() yields:
    named tuples (or regular tuples)

Advantages:
    • Much faster than iterrows
    • Attribute access (row.colname)
    • Less memory overhead
"""

for row in df.itertuples(index=True):
    print("Index:", row.Index)
    print("Country:", row.country)
    print("Score:", row.happiness_score)

# Without index (often cleaner)
for row in df.itertuples(index=False):
    print(row.country, row.happiness_score)

# ✅ Preferred iteration method if iteration is unavoidable


# ------------------------------------------------------------------------
-----
# 3) Iterating over selected columns only
# ------------------------------------------------------------------------
-----

"""
Often you do NOT need the full row.
Selecting columns first reduces overhead and improves clarity.
"""

for row in df[["country", "happiness_score"]].itertuples(index=False):
    print(row.country, row.happiness_score)

# Same idea with iterrows (slower)
for _, row in df[["country", "happiness_score"]].iterrows():
    print(row["country"], row["happiness_score"])
```

```python
# ------------------------------------------------------------------------
-----
# 4) Iterating over FILTERED rows
# ------------------------------------------------------------------------
-----

"""
Always filter FIRST, then iterate.
Never put `if` logic inside the loop if you can filter beforehand.
"""

high_happiness = df[df["happiness_score"] >= 6]

for row in high_happiness.itertuples(index=False):
    print("High happiness:", row.country, row.happiness_score)


# ------------------------------------------------------------------------
-----
# 5) Using enumerate when index is irrelevant
# ------------------------------------------------------------------------
-----

for i, row in enumerate(df.itertuples(index=False)):
    print(f"Row {i}:", row.country)


# ------------------------------------------------------------------------
-----
# 6) Iterating to BUILD external structures (valid use case)
# ------------------------------------------------------------------------
-----

"""
Iteration is often justified when:
• Building a graph (NetworkX)
• Writing to files
• Calling external APIs
• Complex conditional logic
"""

edges = []
for row in df.itertuples(index=False):
    edges.append((row.country, row.world_region))

print("Edges:", edges)
```

```
# ------------------------------------------------------------------------
-----
# 7) When NOT to iterate (VERY IMPORTANT)
# ------------------------------------------------------------------------
-----

"""
❌ BAD (slow, non-idiomatic):
"""

results = []
for _, row in df.iterrows():
    results.append(row["happiness_score"] * 2)

"""
✅ GOOD (vectorized):
"""

results = df["happiness_score"] * 2


# ------------------------------------------------------------------------
-----
# 8) Comparing row-iteration methods
# ------------------------------------------------------------------------
-----

"""
Method         | Speed | Row Type         | Modifies df?
-----------------------------------------------------
iterrows      | slow  | Series           | ❌ no
itertuples    | fast  | namedtuple       | ❌ no
apply(axis=1)| slow  | Series           | ❌ no
vectorized    | fast  | Series/ndarray   | ✅ yes
"""



# ------------------------------------------------------------------------
-----
# 9) Mental model (REMEMBER THIS)
# ------------------------------------------------------------------------
-----

"""
• iterrows():
    "Give me each row as a pandas Series"
• itertuples():
    "Give me each row as a lightweight tuple"
• Vectorization:
    "Do it for all rows at once — this is what pandas wants"
```

```
Rule of thumb:
--------------
✔ Use vectorized ops whenever possible
✔ If iterating, use itertuples()
✔ Filter columns and rows BEFORE iterating
✔ Never modify the row object expecting df to change
"""
```

```

```

```

```